OBJECT ORIENTED PROGRAMMING SYSTEM(oops)

Major principles of object oriented programming system are given below:

- **classes and objects**
- **encapsulation**
- **abstraction**
- **inheritance**
- **polymorphism**

class:

class is a blue print.
(or)
class which doesnot exist physically in the real world
(or)
class is combination of group of objects.

object:

An object is instance of class.
(or)
An object which can exits physically in the real world.

Example:
car(class)

Audi
benz
BMW
Tata
innova
jeep
Mahindra

Syntax:

```
class  class_name:
      <statements>
     ……————————
    ……————————————.
   <statements-n>
object_name=class_name()
```

‘self’ parameter:

it is a reference to the current instance of the class, and it is used to access variables that belongs to the class.

note:

we can create any no.of objects to a single class name.

Example-1:

```
class person:
   def name(self):
      print("my name is john")
   def desg(self):
      print("software trainer")
p=person()#'p' is nothing but object to the person class
p.name()#by using 'p' object we can call methods which are present in class
p.desg()
```

Method:

it is a function that is associated with an object. In python, method is not unique to class instance.

Any object type can have methods.

inheritance:

creating a new class from already existing class is called inheritance.

polymorphism:

poly means ‘many’, morphism means ‘forms’.

it is derived from latin and greek words

the ability to exists in many forms is called polymorphism

in programming, if the same method or variable or object performs various tasks is called polymorphism.

Encapsulation:

Taking data and methods into single unit is called encapsulation

A class is an example for encapsulation.

the main advantage of encapsulation is that it is possible to use same names for the members of two different classes.

Abstraction:

it means hiding of data
hiding of unnecessary data from the user is called abstraction.
it is used to hide internal details and show only functionalities.

difference between method and function

Method	Function
Associated with objects	Not associated with any object.
Cannot invoke by its name	Can invoke by its name
Dependent on class	independent
Require 'self'	Do not require 'self'